

Sintaxe da Linguagem JSX

Objetivo da Aula

O principal objetivo da aula é capacitar o aluno a compreender a sintaxe da linguagem JSX, no contexto do React, bem como conhecer as regras e considerações especiais, para escrever código JSX corretamente, e aplicar a sintaxe JSX na criação de interfaces dinâmicas, utilizando a renderização condicional e a renderização de listas.

Apresentação

Seja bem-vindo à aula de **sintaxe da linguagem JSX**, uma parte essencial para compreender o React e a sua sintaxe única para criação de interfaces. Nesta jornada de aprendizado, nosso objetivo é familiarizá-lo com a sintaxe JSX, essencial para o desenvolvimento front-end.

Iniciaremos abordando a sintaxe JSX do React, uma extensão do JavaScript que permite escrever código HTML, de forma declarativa, dentro do JavaScript. Veremos, também, como ela é traduzida em JavaScript.

Além disso, exploraremos algumas considerações especiais e regras da sintaxe JSX. Entenderemos as particularidades do JSX, como a necessidade de envolver o código em uma única tag ou o uso adequado de chaves para inserção de expressões JavaScript.

Por fim, abordaremos como realizar renderização condicional e renderização de listas. Essas habilidades são cruciais para a construção de componentes reutilizáveis que exibam informações de maneira organizada e escalável.

1. Introdução ao JSX

O JSX, conhecido também como JavaScript XML, é uma extensão de sintaxe do JavaScript que permite a criação de elementos da interface do usuário, de forma **declarativa**, possibilitando ao JavaScript declarar elementos JSX, similar ao HTML (Minnick, 2022). Essa sintaxe foi criada pelo Facebook e é comumente usada em conjunto com a biblioteca React para a criação de aplicativos web.

Com a evolução de bibliotecas e frameworks para criação de aplicações front-end, muitas alternativas, que também utilizam uma sintaxe JSX, foram criadas, como o Vue.js, Solid.js, Lit e Vanilla (Vue.Js, 2023; Solidjs, 2023; Lit, 2023). O objetivo principal dessa sintaxe é isolar do código as declarações HTML do JavaScript, dentro do próprio JavaScript. Isso torna o desenvolvimento de interfaces mais fácil e compreensível para muitos desenvolvedores.

Por outro lado, a sintaxe JSX não é nativamente compreendida pelos interpretadores JavaScript, que estão embutidos nos navegadores. Os interpretadores compreendem apenas códigos em JavaScript. Por esses motivos, um processo conhecido por **transpilação** é utilizado para traduzir o código escrito, com a linguagem JSX, para JavaScript.



Destaque

Ao longo desta aula, utilize a ferramenta on-line Babel, disponível em <https://babeljs.io/repl> (acesso em 19 set. 2023.), para compreender o processo de transpilação da sintaxe JSX do React para JavaScript.

2. A Sintaxe JSX do React

A transpilação de JSX para JavaScript é necessária, pois o navegador não compreende JSX. Conforme Minnick (2022), antes da versão 17 do React, a sintaxe JSX foi construída dentro do módulo react, sendo transpilada para JavaScript, utilizando a função `React.createElement()`. Após a versão 17, foi retirada a necessidade do módulo react, sendo agora o JSX transpilado para JavaScript com a função `jsx()`.



Você Sabia?

Transpiladores são um tipo especial de compilação que traduz o código-fonte de uma linguagem em outra (Fuertes, 2023). Babel e TypeScript são exemplos de compiladores código-para-código ou, melhor dizendo, transpiladores.



Destaque

Antes e depois da versão 17 do React, não houve mudanças na sintaxe JSX, mas sim em como o JSX passou a ser traduzido nos transpiladores, como o Babel e o TypeScript.

Para melhor experiência com os exemplos seguintes, ao utilizar a ferramenta on-line Babel, você pode obter o resultado da transpilação do JSX do React para JavaScript, na versão antiga, mudando a opção do item React Runtime para “Classic”, ou, na versão atual, mudando a opção para “Automatic”.

Na versão antiga, a tradução do código, com a sintaxe JSX, será *React.createElement(type, props,...children)*, em que:

- **type**: representa o tipo de elemento HTML a ser declarado;
- **props**: representa um objeto com as propriedades do elemento HTML, exemplo: `{ id: "hello", className: "Titulo1" }`;
- **children**: representa 0 ou mais elementos-filho que um elemento HTML possa conter. A declaração de cada elemento-filho também deve ser realizada com a função *createElement*.

Por exemplo, vamos supor que o seu código seja apresentado a seguir.

```
import React from 'react';  
function App() {  
  return <h1>Hello World</h1>;  
}  
export default App;
```

Assim, a declaração `<h1>Hello World</h1>` é traduzida em:

```
import React from 'react';  
function App() {  
  return React.createElement('h1', null, 'Hello world');  
}  
export default App;
```

A partir da versão 17, a linha *import React from 'react'* não é mais necessária, e a transpilação utilizará a função *jsx(type, props, key)*, em que:

- **type**: representa o tipo de elemento HTML a ser declarado;
- **props**: representa um objeto, com as propriedades do elemento HTML, além de elementos-filho, exemplo: `{ id: "Lista1", children: jsx(...) }`;
- **key**: representa uma propriedade especial, que deve ser utilizada separadamente de props. É, geralmente, utilizada em elementos-filho de uma lista HTML. O seu objetivo é identificar unicamente cada elemento, permitindo ao React identificar quais elementos foram adicionados, mudados ou removidos.

Assim, a declaração `<h1>Hello World</h1>` é traduzida em:

```
import {jsx as _jsx} from 'react/jsx-runtime';  
function App() {  
  return _jsx('h1', { children: 'Hello world' });  
}  
export default App;
```

A seguir, temos um exemplo de um código com a sintaxe JSX, contendo elementos que possuem propriedades e elementos-filho.

```
<ul id="Lista1" className="Lista">  
  <li key="item1">Item 1</li>  
  <li key="item2">Item 2</li>  
</ul>
```

No transpilação antiga, a sintaxe JSX acima é traduzida em:

```
React.createElement( // elemento raiz  
  "ul", // type  
  { id: "Lista1", className: "Lista" }, // props  
  React.createElement( // primeiro elemento filho  
    "li", // type
```

```

    { key: "item1" }, // props
    "Item 1" // elemento filho (innerHTML)
  ),
  React.createElement( // segundo elemento filho
    "li", // type
    { key: "item2" }, // props
    "Item 2" // elemento filho (innerHTML)
  )
);

```

A partir do React 17, o trecho de código em JSX é traduzido em:

```

import { jsx as _jsx } from "react/jsx-runtime";
import { jsxs as _jsxs } from "react/jsx-runtime";
_jsxs( // elemento raiz
  "ul", // type
  { // props
    id: "Lista1",
    className: "Lista",
    children: [ // lista de filhos
      _jsx( // primeiro filho
        "li", // type
        { children: "Item 1" /* filho (innerHTML) */, // propos
          "item1" // key
        }, _jsx( // segundo filho
          "li", // type
          { children: "Item 2" /* filho (innerHTML) */, // propos
            "item2" // key
          }
        )
      ]
    }
  );

```

3. Considerações Especiais e Regras da Sintaxe JSX

Conforme Minnick (2022), a sintaxe JSX é uma sintaxe baseada em XML, o que também requer conformidade com as regras XML. Além do mais, algumas regras e particularidades, apontadas por Minnick (2022), são apresentadas a seguir.

- Assim como em HTML, todos os elementos deverão ser fechados.
- Elementos que não possuem elementos-filho também deverão ser fechados, como as tags `br`, `input` e `link`. Em contraste, no padrão HTML5, é comum a tag `br` não ser fechada.
- Elementos HTML escritos em JSX deverão ser todos escritos em minúsculo, devido à sintaxe JSX compreender elemento que começa com maiúscula como um componente do React – por exemplo, `App`. Em contraste, documentos HTML não distinguem tags maiúsculas de minúsculas.
- Palavras reservadas do JavaScript não podem ser utilizadas como propriedades de elementos em JSX, uma vez que esses elementos serão traduzidos para JavaScript. Por exemplo, `class` e `for` são propriedades comuns em elementos HTML, porém são palavras reservadas da sintaxe JavaScript. Como solução, `class` em HTML se torna `className` em JSX, e `for` em HTML se torna `htmlFor` em JSX.
- Nomes de atributos em JSX, formados por mais de uma palavra, são escritos em **camelCase**, exemplos: `className`, `tabIndex` e `onClick`.
- Pares de chaves são utilizados caso se deseje declarar expressões, literais e variáveis, dentro da linguagem JSX, como:
 - `<p className={variavel1}>{variavel2}</p>`,
 - `<div>{item.disponivel? <p>{item.nome}</p>: null}</div>`

Conforme a documentação oficial do React (React.Dev, 2023), cada componente react deverá retornar um único elemento-raiz em JSX.

A exemplo, o trecho de código a seguir apresentará erros de execução.

```
export default function App() {
  return <p>Parágrafo 1</p>
  <p>Parágrafo 2</p>;
}
```

Como solução, você pode utilizar a tag **div** como o elemento raiz, e os dois parágrafos **p** como elementos filho de **div**.

```
export default function App() {
  return <div>
    <p>Parágrafo 1</p>
    <p>Parágrafo 2</p>;
  </div>
}
```

O grande problema é que não há necessidade de o elemento **div** ser apresentado na interface. Por esses motivos, o React criou um elemento vazio, conhecido como **Fragment**. Elementos **Fragment** permitem agrupar elementos HTML sem deixar rastros de sua existência na estrutura HTML. O elemento vazio pode ser declarado de duas formas:

1. **<React.Fragment>... </React.Fragment>**, que requer importação do módulo **react**.
2. **<>... </>**, que não necessita da importação do módulo **react**.

Assim, a melhor alternativa é utilizar a tag vazia, como segue.

```
export default function App() {
  return <>
    <p>Parágrafo 1</p>
    <p>Parágrafo 2</p>;
  </>
}
```

4. Renderização Condicional

Como a sintaxe da linguagem JSX é **declarativa**, você deve especificar o que deseja alcançar, mas não necessariamente como fazer. Assim, estruturas condicionais, como blocos **if/else** e **switch/case**, não são suportados em JSX.

4.1. Renderização Condicional com Blocos if/else

Caso deseje realizar alguma lógica estruturada, você pode estruturar condições com blocos if/else, como apresentado a seguir.

```
export default function App() {  
  const tarefa = { id: "T1", descricao: "Tarefa 1", finalizada: true };  
  if(tarefa.finalizada) {  
    return <p>{ tarefa.descricao + " " }</p>;  
  } else {  
    return <p>{tarefa.descricao}</p>;  
  }  
}
```

4.2. Renderização com Operador de Condição Ternária em JSX

Como a linguagem JavaScript possui um operador de atribuição com condição ternária, é possível utilizá-lo diretamente na declaração de elementos JSX. Assim, podemos traduzir o bloco if/else, do código anterior, conforme apresentado a seguir.

```
export default function App() {  
  const tarefa = { id: "T1", descricao: "Tarefa 1", finalizada: true };  
  return <p>  
    {tarefa.finalizada  
      ? tarefa.descricao + " "  
      : tarefa.descricao  
    }  
  </p>  
}
```




Destaque

O operador de condição ternária apresenta a seguinte sintaxe: `<condição>? <valor_cond_verdadeira>: <valor_cond_falsa>`.

4.3. Renderização Condicional Retornando Nada com Null

Em JSX, você pode utilizar *null* nas situações em que você não deseja renderizar nada em particular, conforme o trecho de código a seguir.

```
export default function App() {
  const tarefa = { id: "T1", descricao: "Tarefa 1", finalizada: true };
  return tarefa.finalizada? null: <p>{tarefa.descricao}</p>;
}
```

4.4. Renderização Condicional com Operador Lógico && (E)

Em JavaScript, o operador && lógico é verdadeiro quando os operandos à esquerda e à direita forem ambos verdadeiros, sendo a expressão avaliada da esquerda para a direita, respectivamente. Se ambos os operandos forem verdadeiros, então o operador && irá retornar o operando à direita.

Assim, você pode retornar, condicionalmente, o operando à direita, se aquele à esquerda for verdadeiro. O código a seguir apresenta um exemplo com &&.

```
function App() {
  const tarefa = { id: "T1", descricao: "Tarefa 1", finalizada: true };
  return tarefa.finalizada && <p>{tarefa.descricao}</p>;
}
export default App;
```

5. Renderização de Listas

Frequentemente, você precisará exibir elementos similares de uma coleção de dados. Para isso, você pode utilizar funções de lista ***filter()***, para filtrar os dados da coleção, e ***map()***, para transformar os dados em elementos HTML.

5.1. Transformando Dados de Lista em Elementos HTML

Considere que você possui uma lista de tarefas com as propriedades `id` e `descricao` do tipo string, sendo `finalizada` do tipo boolean. Você deseja exibir cada tarefa como item (`<li>`) de uma lista (`<ul>`). O trecho de código a seguir apresenta uma solução.

```
const tarefas = [
  { id: "T1", descricao: "Tarefa 1", finalizada: true },
  { id: "T2", descricao: "Tarefa 2", finalizada: true },
  { id: "T3", descricao: "Tarefa 3", finalizada: false },
  { id: "T4", descricao: "Tarefa 4", finalizada: true },
];

export default function App() {
  return <ul>
  {
    tarefas.map(tarefa =>
      <li key={tarefa.id}>
        {tarefa.descricao}
      </li>
    )
  }
  </ul>;
}
```

Observe que a função ***map()*** transforma cada tarefa em uma em uma lista de itens HTML e apresenta ao navegador o seguinte resultado:

- Tarefa 1
- Tarefa 2
- Tarefa 3
- Tarefa 4

Além do mais, o React exige que cada item da lista seja único, e, para isso, devemos atribuir à propriedade especial **key** um identificador único; caso contrário, a seguinte mensagem de alerta será apresentada no console:

- **Warning: Each child in a list should have a unique “key” prop.**

Opcionalmente, você também pode filtrar os dados antes de traduzi-los em elementos HTML. Considere, ainda, o mesmo cenário do exemplo anterior. Agora, desejamos criar dois tipos de listas: uma lista de tarefas pendentes, e outra lista de tarefas concluídas. Como solução, você pode desenvolver o seguinte código:

```
const tarefas = [  
  { id: "T1", descricao: "Tarefa 1", finalizada: true },  
  { id: "T2", descricao: "Tarefa 2", finalizada: true },  
  { id: "T3", descricao: "Tarefa 3", finalizada: false },  
  { id: "T4", descricao: "Tarefa 4", finalizada: true },  
];  
  
export default function App() {  
  const tarefasConcluidas = tarefas.filter(tarefa => tarefa.finalizada);  
  const tarefasPendentes = tarefas.filter(tarefa => !tarefa.finalizada);  
  return <>  
    <h1>Tarefas Pendentes:</h1>  
    <ul>{  
      tarefasPendentes  
        .map(tarefa => <li key={tarefa.id}>{tarefa.descricao}</li>)  
    }</ul>  
  </>  
}
```

```

    <h1>Tarefas Concluídas:</h1>

    <ul>{
      tarefasConcluidas
      .map(tarefa => <li key={tarefa.id}>{tarefa.descricao}</li>)
    }</ul>

  </>;
}

```

Após a renderização do trecho de código acima, no navegador, a seguinte interface será apresentada ao usuário:

Tarefas Pendentes:

- Tarefa 3

Tarefas Concluídas:

- Tarefa 1
- Tarefa 2
- Tarefa 4

Considerações Finais da Aula

Ao término desta aula, você possui uma base sólida para explorar a sintaxe JSX e criar interfaces de forma inteligente.

Inicialmente, aprendemos sobre a sintaxe JSX do React e como ela nos permite combinar o HTML com o JavaScript de forma declarativa. Essa abordagem torna a escrita de código mais intuitiva e facilita a visualização da estrutura da interface.

Compreendemos, também, as principais considerações e regras da sintaxe, que são fundamentais para evitar erros e ambiguidades em nossos componentes. A familiarização com essas regras nos permite escrever código JSX coeso e de fácil manutenção.

Ao explorar a renderização condicional, aprendemos a criar interfaces dinâmicas, adaptando o conteúdo, com base em condições lógicas. Essa habilidade é essencial para desenvolver aplicações que respondam às diferentes situações e necessidades dos usuários.

Além disso, abordamos a renderização de Listas, aprendendo a criar elementos JSX dinamicamente a partir delas. Essa técnica nos permite exibir informações de forma organizada e escalável.

Ao concluir esta aula, tenha a certeza de que o domínio do JSX abrirá portas para um desenvolvimento mais produtivo e eficiente no React.

Material Complementar



HTML to JSX (transform.tools)

2023, Transform.

Para facilitar o aprendizado da sintaxe JSX do React, você pode transformar elementos HTML em elementos JSX e vice-versa. Isso facilitará a sua compreensão de detalhes a respeito da sintaxe.

Link para acesso: <https://transform.tools/html-to-jsx> (acesso em 18 set. 2023.)

Referências

BABEL. Disponível em: <https://babeljs.io>. Acesso em: 1 jul. 2023.

FUERTES, Andrés Bastidas; PÉREZ, María; HORMAZA, Jaime Meza. Transpilers: A Systematic Mapping Review of Their Usage in Research and Industry. **Appl. Sci.**, 13(6), 3667, 2023.

LIT. Disponível em: <https://lit.dev>. Acesso em: 1 jul. 2023.

REACT.DEV. **Writing Markup with JSX**. React.dev, 2023. Disponível em: <https://react.dev/learn/writing-markup-with-jsx>. Acesso em: 1 jul. 2023.

RUAN, Luna. **Introducing the New JSX Transform**. React: A JavaScript library for building user interfaces. REACTJS, 22 set. 2020. Disponível em: <https://legacy.reactjs.org/blog/2020/09/22/introducing-the-new-jsx-transform.html>. Acesso em: 1 jul. 2023.

SOLIDJS. Disponível em: <https://solidjs.com>. Acesso em: 1 jul. 2023.

VUE.JS. Disponível em: <https://vuejs.org>. Acesso em: 1 jul. 2023.